

DATA STRUCTURES & ALGORITHMS LAB

Lab Assignment 1 — Part 1: ArrayList, Pointer Traversal & Problem Solving

Course	CSC211 — Data Structures & Algorithms Lab	Total Marks	5
Time Allowed	1 Hour 30 Minutes	Programming Language	C++
CLO	CLO-4 — Implement data structures and algorithms	Bloom Level	Applying

Student Name: Hafsa Maryam Registration No.: FA25-BCS-047

Problem Statement

Develop a C++ program that implements and analyzes an ArrayList using a struct. The ArrayList has a fixed capacity, a logical size, and operations for insertion and deletion. A single pointer named ptr will be used to traverse the logical elements stored in the ArrayList.

Starter Code

```
const int CAPACITY = 20;

struct ArrayList
{
    int data[CAPACITY];
    int size = 0;
};

ArrayList list;
int *ptr = list.data;      // pointer used to traverse the ArrayList

int minimum;
int maximum;
int median;
int closestValue;
int closestPosition;
int sum = 0;

double generalAverage;
double specialAverage;
double averageDifference;
double finalScore;
```

You may implement the following functions:

```
bool insertEnd(ArrayList &list, int value);
bool insertAtBeginning(ArrayList &list, int value);
bool deleteAtPosition(ArrayList &list, int position);
void displayList(const ArrayList &list);
```

Part A — Create and Populate the ArrayList (0.5 Mark)

Insert the following nine values into the ArrayList using insertEnd():

```
18, 7, 45, 11, 36, your_reg_no, 21, 13, 29
```

- Replace your_reg_no with the last two digits of your own registration number. Example: a registration number ending in 08 uses the value 8.
- Do not directly initialize data[] with the nine values; add them using the ArrayList insertion operation.
- After insertion, list.size must be 9. Display the initial ArrayList.

Part B — Pointer Traversal, Sum, Minimum and Maximum (1 Mark)

Set ptr = list.data and use ptr to traverse only the logical elements of the ArrayList. During the same traversal:

- calculate the sum of all logical elements;
- determine the minimum value;
- determine the maximum value;
- use pointer movement and dereferencing (for example, *ptr and ptr++).

Do not perform a separate traversal solely to calculate the sum. Display Minimum Value, Maximum Value and Sum.

Part C — Find the Median (1 Mark)

The original ArrayList must not be rearranged. Create a temporary array containing only the logical elements, sort the temporary array in ascending order using a sorting technique implemented by you, and determine the middle value.

- Do not use sort() or another STL sorting algorithm.
- Since the ArrayList initially contains nine values, store the middle value in median.
- Display Median Value.

Part D — Two Averages and Closest Value (1 Mark)

General Average = Sum / list.size

Special Average = (Minimum + Median + Maximum) / 3

Reset ptr = list.data and traverse the ArrayList again. Find the element having the smallest absolute distance from specialAverage. Store that element in closestValue and its logical position in closestPosition.

- Distance = |Value - Special Average|.
- If two values are equally close, retain the first value encountered.
- Display General Average, Special Average, Closest Value and Position of Closest Value.

Part E — Final Calculations and ArrayList Modification (1 Mark)

Average Difference = |General Average - Special Average|

Final Score = |Closest Value - General Average| + |Closest Value - Special Average| + Average Difference

- Display Difference Between Averages and Final Score.
- Delete the element at closestPosition using deleteAtPosition(). Shift elements correctly and update list.size.
- Round specialAverage to the nearest integer and insert it at the beginning using insertAtBeginning(). Shift elements correctly and update list.size.
- Display the ArrayList after deletion and again after insertion.

Part F — Code Understanding & Verification (0.5 Mark)

You must be able to explain and modify your submitted program. The instructor may ask you to explain what ptr and *ptr represent, why traversal stops at list.size rather than CAPACITY, why insertion/deletion require shifting, why a temporary array is used for the median, or may change one value/operation and ask you to modify the program.

Required Program Output

```
Initial ArrayList:  
Minimum Value:  
Maximum Value:  
Median Value:  
Sum:  
General Average:  
Special Average:  
Closest Value:  
Position of Closest Value:  
Difference Between Averages:  
Final Score:  
ArrayList After Deletion:  
Final ArrayList After Insertion:
```

Programming Restrictions

1. The solution must be written in C++ and must use the provided struct ArrayList.
2. The nine initial values must be inserted using insertEnd(); size represents the number of logical elements currently stored.
3. Do not access unused positions between size and CAPACITY.
4. ptr is the single pointer used to traverse the ArrayList in the required sections.
5. Do not use vector, list, or another STL container.
6. Do not use sort(), min_element(), max_element(), or similar STL algorithms.
7. Insertion and deletion must be implemented manually using shifting.
8. Use the last two digits of your own registration number.
9. Correct numerical output alone is not sufficient; the required ArrayList implementation and pointer traversal must be demonstrated.

AI Tools, Code Sharing and Originality

Students are responsible for understanding every line of code they submit, irrespective of whether assistance is obtained from ChatGPT, another AI system, an online source, or another person.

If two or more submissions contain identical or substantially similar solutions, including substantially identical AI-generated solutions, the first timestamped submission will be considered the original submission. Later matching submissions will be subject to verification and may not receive credit if the student cannot demonstrate ownership and understanding of the submitted code.

The instructor may conduct a short viva or request an immediate modification to the program.

Mark Distribution

Component	Marks
Create & Populate ArrayList	0.5
Pointer Traversal, Sum, Minimum & Maximum	1.0
Median & Manual Sorting	1.0
Averages & Closest Value	1.0
Final Calculations & ArrayList Modification	1.0
Code Understanding & Verification	0.5

Total	5.0
-------	-----

Code:

```
#include <iostream>
using namespace std;

// --- Starter Code ---
const int CAPACITY = 20;

struct ArrayList {
    int data[CAPACITY];
    int size = 0;
};

// Global variables
ArrayList list;
int *ptr = list.data;

int minimum, maximum, median, closestValue, closestPosition, sum = 0;
double generalAverage, specialAverage, averageDifference, finalScore;

// 1. Insert at End
bool insertEnd(ArrayList &list, int value) {
    if (list.size >= CAPACITY) return false;
    list.data[list.size] = value;
    list.size++;
    return true;
}

// 2. Insert at Beginning (Shifts elements to the right)
bool insertAtBeginning(ArrayList &list, int value) {
    if (list.size >= CAPACITY) return false;
    // Start from the end and move everything one step right
    for (int i = list.size; i > 0; i--) {
        list.data[i] = list.data[i - 1];
    }
}
```

```
list.data[0] = value;
list.size++;
return true;
}
```

// 3. Delete at Position (Shifts elements to the left)

```
bool deleteAtPosition(ArrayList &list, int position) {
    if (position < 0 || position >= list.size) return false;
    // Start from the position and move everything one step left
    for (int i = position; i < list.size - 1; i++) {
        list.data[i] = list.data[i + 1];
    }
    list.size--;
    return true;
}
```

// 4. Display List

```
void displayList(const ArrayList &list) {
    for (int i = 0; i < list.size; i++) {
        cout << list.data[i] << " ";
    }
    cout << endl;
}
```

```
int main() {
```

```
    cout << "--- Part A ---" << endl;
```

```
    insertEnd(list, 18);
    insertEnd(list, 7);
    insertEnd(list, 45);
    insertEnd(list, 11);
```

```
insertEnd(list, 36);  
insertEnd(list, 47);  
insertEnd(list, 21);  
insertEnd(list, 13);  
insertEnd(list, 29);
```

```
cout << "Initial ArrayList:" << endl;  
displayList(list);  
cout << endl;
```

```
cout << "--- Part B ---" << endl;
```

```
ptr = list.data; // Reset pointer to start
```

```
minimum = *ptr;  
maximum = *ptr;  
sum = 0;
```

```
int *minPtr = list.data;  
int *maxPtr = list.data;
```

```
for (int i = 0; i < list.size; i++) {  
    int currentValue = *ptr; // Get value from pointer
```

```
    sum = sum + currentValue;
```

```
    if (currentValue < minimum) {  
        minimum = currentValue;  
        minPtr = ptr; // Save address of new minimum  
    }
```

```
    if (currentValue > maximum) {  
        maximum = currentValue;  
        maxPtr = ptr; // Save address of new maximum
```

```
}

ptr++; // Move pointer to next element
}

cout << "Minimum Value: " << minimum << endl;
cout << "Maximum Value: " << maximum << endl;
cout << "Sum: " << sum << endl;

cout << "--- Part C ---" << endl;

// Copy to temporary array
int tempArr[CAPACITY];
for (int i = 0; i < list.size; i++) {
    tempArr[i] = list.data[i];
}

// Simple Bubble Sort
for (int i = 0; i < list.size - 1; i++) {
    for (int j = 0; j < list.size - i - 1; j++) {
        if (tempArr[j] > tempArr[j + 1]) {
            int temp = tempArr[j];
            tempArr[j] = tempArr[j + 1];
            tempArr[j + 1] = temp;
        }
    }
}

// The middle element of 9 items is at index 4
median = tempArr[list.size / 2];

// Find where this median value is in the original list and save its pointer
int *medianPtr = list.data;
for (int i = 0; i < list.size; i++) {
    if (list.data[i] == median) {
        medianPtr = &list.data[i];
    }
}
```

```
        break;
    }
}

cout << "Median Value: " << median << endl;

cout << "--- Part D ---" << endl;

// 1. General Average
generalAverage = (double)sum / list.size;
cout << "General Average: " << generalAverage << endl;

// 2. Special Average (using saved pointers)
specialAverage = (double)(*minPtr + *medianPtr + *maxPtr) / 3.0;
cout << "Special Average: " << specialAverage << endl;

// 3. Find Closest Value
ptr = list.data; // Reset pointer to start
int *closestPtr = list.data;
double smallestDistance = 1000000.0; // Start with a very large number

for (int i = 0; i < list.size; i++) {
    double distance;

    // Manual absolute value calculation (no cmath library needed)
    if (*ptr > specialAverage) {
        distance = *ptr - specialAverage;
    } else {
        distance = specialAverage - *ptr;
    }

    // If this distance is smaller than the previous smallest, update it
    if (distance < smallestDistance) {
        smallestDistance = distance;
        closestPtr = ptr;
    }
}
```

```
ptr++; // Move pointer
}

closestValue = *closestPtr;
// Pointer math to find the position (address difference)
closestPosition = closestPtr - list.data;

cout << "Closest Value: " << closestValue << endl;
cout << "Position of Closest Value: " << closestPosition << endl;
cout << "--- Part E ---" << endl;

// 1. Calculations
if (generalAverage > specialAverage) {
    averageDifference = generalAverage - specialAverage;
} else {
    averageDifference = specialAverage - generalAverage;
}

// Final Score formula
double diff1, diff2;
if (*closestPtr > generalAverage) diff1 = *closestPtr - generalAverage;
else diff1 = generalAverage - *closestPtr;

if (*closestPtr > specialAverage) diff2 = *closestPtr - specialAverage;
else diff2 = specialAverage - *closestPtr;

finalScore = diff1 + diff2 + averageDifference;

cout << "Difference Between Averages: " << averageDifference << endl;
cout << "Final Score: " << finalScore << endl;

// 2. Delete the closest value
closestPosition = closestPtr - list.data; // Recalculate position just to be safe
deleteAtPosition(list, closestPosition);

cout << "ArrayList After Deletion:" << endl;
```

```
displayList(list);

// 3. Insert rounded Special Average at beginning
// Adding 0.5 then casting to int does the rounding for positive numbers
int roundedAvg = (int)(specialAverage + 0.5);
insertAtBeginning(list, roundedAvg);

cout << "Final ArrayList After Insertion:" << endl;
displayList(list);

return 0;
}
```

Output:

```
--- Part A ---
Initial ArrayList:
18 7 45 11 36 47 21 13 29

--- Part B ---
Minimum Value: 7
Maximum Value: 47
Sum: 227
--- Part C ---
Median Value: 21
--- Part D ---
General Average: 25.2222
Special Average: 25
Closest Value: 21
Position of Closest Value: 6
--- Part E ---
Difference Between Averages: 0.222222
Final Score: 8.444444
ArrayList After Deletion:  18 7 45 11 36 47 13 29
Final ArrayList After Insertion: 25 18 7 45 11 36 47 13 29

-----
Process exited after 0.2233 seconds with return value 0
Press any key to continue . . . |
```